USBasp-Pico

라즈베리파이 피코 (RP2040) 용
USBasp 호환 AVR 프로그래머

TinyUSB Vendor-Class + ISP/TPI/HVSP 백엔드
5V / 3.3V AVR 지원 + 12V HVSP 모드

버전 1.9 / 2026-11-12
Z.ai

목차

목차 2

1. 프로젝트 개요 3

2. USBasp 프로토콜 분석 4

3. RP2040 펌웨어 아키텍처 5

4. 하드웨어 구성 및 핀 할당 6

5. 배선도 7

5.1 5V AVR 타겟 + 레벨시프터 7

5.2 3.3V AVR 타겟 직접 연결 7

5.3 12V HVSP 모드 (ATtiny85 복구) 8

6. 12V HVSP 모드 상세 9

6.5 SWIO 모드 (CH32V003 RISC-V) 11

7. 빌드 방법 14

8. avrdude 호환성 및 사용법 15

9. 테스트 및 검증 16

10. 관련 프로젝트 및 참고자료 17

11. 라이선스 및 크레딧 18

1. 프로젝트 개요

USBasp-Pico는 라즈베리파이 피코 (RP2040)를 기반으로, 원본 USBasp (Thomas Fischl 저, mariusgreuel 포크 기반)와 완벽하게 호환되는 AVR 프로그래머 펌웨어입니다. 호스트 PC의 avrdude은 일반 USBasp로 인식하며, 별도의 드라이버나 설정 변경 없이 `-c usbasp` 옵션으로 사용할 수 있습니다.

원본 USBasp는 ATmega8 + V-USB(소프트웨어 USB, 저속 1.5 Mbps) 조합으로 구성되어 있으나, RP2040은 하드웨어 USB 풀스피드 (12 Mbps) 컨트롤러와 듀얼 코어 Cortex-M0+를 갖추고 있어 훨씬 빠르고 안정적인 통신이 가능합니다. 또한 플래시 2MB, RAM 264KB로 고용량 AVR (256KB 이상 플래시) 프로그래밍에도 제약이 없습니다.

주요 특징

- USBasp 완벽 호환 — VID=0x16C0, PID=0x05DC, 동일 프로토콜
- 고용량 AVR 지원 — ATmega2560 등 256KB 플래시 칩까지 (SETLONGADDRESS 지원)
- TPI 모드 — ATtiny 4/5/9/10/20/40 등 소형 AVR 프로그래밍
- HVSP 12V 모드 (옵션) — RESET 비활성화된 ATtiny85 등 복구용
- SCK 자동/수동 설정 — 500 Hz ~ 1.5 MHz 12단계
- UF2 자동 빌드 — Pico BOOTSEL 모드로 드래그 앤 드롭 플래싱
- 확장 핀 할당 — ISP + HVSP 동시 핀 매핑, LED 상태 표시

본 프로젝트는 mariusgreuel/USBasp 포크의 프로토콜 사양을 그대로 유지하면서, 구현을 RP2040 + TinyUSB로 옮긴 것입니다. 원본 V-USB 기반 ATmega8 펌웨어와 호환되는 모든 avrdude 기능을 그대로 사용할 수 있습니다.

2. USBasp 프로토콜 분석

USBasp는 V-USB 기반의 ATmega8 펌웨어로, USB low-speed (1.5 Mbps)에서 동작합니다. 모든 명령은 EP0 vendor-specific control transfer로 전달되며, bmRequestType은 0x40 (OUT) 또는 0xC0 (IN), bRequest는 함수 코드(1~127)로 구성됩니다. wValue와 wIndex는 명령 파라미터를 전달하고, 데이터 스테이지는 블록 전송(READFLASH/WRITEFLASH 등)에 사용됩니다.

USBtinyISP 대비 USBasp 장점

항목	USBtinyISP	USBasp (본 프로젝트)
고용량 AVR (>128KB)	미지원 (1 byte addr)	지원 (SETLONGADDRESS)
USB 속도	저속 1.5 Mbps	풀스피드 12 Mbps
TPI 모드	미지원	지원 (USBASP_FUNC_TPI_*)
블록 전송	EP0 vendor only	EP0 데이터 스테이지 4KB
SCK 단계	8 단계	12 단계 (500 Hz~1.5 MHz)
avrdude 지원	-c usbtiny	-c usbasp (기본)

표 1. USBtinyISP vs USBasp 비교

mariusgreuel 포크는 원본 USBasp와 프로토콜 레벨에서 동일합니다. 다만 윈도우 드라이버 패키징, 빌드 시스템 정리, 일부 USB 설명자 개선이 추가되어 있으며, 본 프로젝트는 이 포크의 프로토콜 상수를 그대로 사용합니다. 확장된 명령어(USBASP_FUNC_TPI_*, USBASP_FUNC_GETCAPABILITIES)도 모두 구현되어 avrdude의 TPI 프로그래밍을 그대로 지원합니다.

3. RP2040 펌웨어 아키텍처

펌웨어는 TinyUSB vendor-class 드라이버 위에 USBasp 프로토콜을 구현하고, 하위 백엔드로 ISP / TPI / HVSP 세 가지 모드를 분리한 구조입니다. USB 통신은 RP2040의 하드웨어 USB 컨트롤러가 처리하며, GPIO bit-bang SPI는 코어0에서 실행됩니다. 코어1은 미사용 상태로 두어 향후 확장 (예: USB CDC 디버그 채널)에 대비합니다.

펌웨어 파일 구조

파일	역할	라인 수
main.c	USB 초기화, LED, 메인 루프	~100
usb_descriptors.c	USB 장치/설정/문자열 디스크립터	~150
usbasp_protocol.c	USBasp vendor-class 프로토콜 핸들러	~280
isp.c / isp.h	ISP bit-bang 백엔드 (500Hz~1.5MHz)	~250
tpi.c / tpi.h	TPI 프로토콜 백엔드 (ATtiny 4/5/9/10)	~150
hvsp.c / hvsp.h	HVSP 12V 모드 백엔드 (ATtiny85 등)	~200
usbasp.h / usbasp_protocol.h	공용 상수/핀 정의	~80
tusb_config.h	TinyUSB 설정 (Vendor 클래스)	~70
CMakeLists.txt	빌드 설정 (Pico SDK + TinyUSB)	~70

표 2. 펌웨어 소스 파일 구조

프로토콜 핸들러는 TinyUSB의 `tud_vendor_control_xfer_cb` 콜백으로 진입합니다. SETUP 단계에서 bRequest를 dispatch하여 ISP/TPI/HVSP 백엔드 함수를 호출하고, 데이터 스테이지는 `tud_control_xfer` 한 번의 호출로 4KB 단위 전송을 처리합니다. 쓰기 명령(WRITEFLASH 등)은 ACK 단계에서 4KB scratch 버퍼의 데이터를 처리합니다.

4. 하드웨어 구성 및 핀 할당

라즈베리파이 피코의 GPIO는 3.3V I/O입니다. 5V AVR 타겟을 프로그래밍하려면 레벨시프터(TXS0108E, 74LVC245 등)가 필요하며, 3.3V AVR은 Pico와 직접 연결이 가능합니다. ISP는 6-pin 표준 커넥터, HVSP는 4개의 신호선 + 12V 제어선으로 구성됩니다. Pico의 온보드 LED(GP25) 대신 GP6을 상태 표시용으로 사용합니다.

ISP 핀 할당 (기본)

Pico 핀	ISP 신호	방향	설명
GP2	SCK	출력	SPI 클럭 (500 Hz ~ 1.5 MHz)
GP3	MOSI / HVSP SDI	출력	Pico -> 타겟 데이터 (HVSP SDI 와 공유, v1.9)
GP4	MISO	입력	타겟 -> Pico 데이터
GP5	RESET	출력	타겟 RESET (활성 LOW)
GP6	LED	출력	활동 표시 LED (활성 LOW)
GND	GND	공통	접지 (타겟과 공통 필수)
3V3	VCC	출력	3.3V AVR 타겟 전원 직접 공급
VBUS	5V	출력	5V AVR 타겟 (레벨시프터経由)

표 3. ISP 핀 할당

HVSP 핀 할당 (옵션, USBASP_PICO_ENABLE_HVSP=1 시) - v1.9

Pico 핀	HVSP 신호	ATtiny85 핀	설명
GP3	SDI (공유)	PB1 (6)	데이터 입력, ISP MOSI 와 동일 핀 공유 (v1.9)
GP7	SCI	PB0 (5)	HVSP 클럭
GP8	SDO	PB2 (7)	데이터 출력 (타겟 -> Pico)
GP9	SII	PB3 (2)	명령어 입력
GP10	12V EN	부스트 ON	12V 부스트컨버터 인에이블
GP11	(free)	-	v1.9 에서 사용자 자유 핀으로 반환

표 4. HVSP 핀 할당 (ATtiny85 기준)

5. 배선도

5.1 5V AVR 타겟 + 레벨시프터

5V AVR (ATmega328P @5V, ATtiny85 @5V 등)를 프로그래밍할 때는 Pico의 3.3V I/O를 5V로 변환해야 합니다. TXS0108E (양방향 자동 방향 전환) 또는 74LVC245 (단방향, MISO/MOSI 방향 고정 필요) 등의 레벨시프터를 사용합니다. 레벨시프터의 HVcc는 타겟 전압(5V)과 같게 설정하고, 타겟 VCC는 Pico VBUS에서 직접 공급합니다.

5.2 3.3V AVR 타겟 직접 연결

3.3V 동작 AVR (ATmega328P @3.3V, ATtiny85 @3.3V 등)은 레벨시프터 없이 Pico와 직접 연결할 수 있습니다. Pico의 3V3 핀에서 타겟 VCC를 공급하고, 모든 신호선은 3.3V 레벨로 직접 연결합니다. Pico의 GPIO는 3.3V tolerant이므로 MISO 입력도 안전합니다.

5.3 12V HVSP 모드 (ATtiny85 복구)

RESET 핀이 RSTDISBL 퓨즈로 비활성화되었거나, 클럭 설정이 잘못되어 일반 ISP 모드로 진입할 수 없는 ATtiny85 (및 ATtiny25/45/13/24/44/84) 칩은 12V HVSP (High-Voltage Serial Programming) 모드로 복구할 수 있습니다. RESET 핀에 12V를 인가하면 백도어 프로그래밍 모드로 진입합니다. 본 펌웨어는 HVSP 백엔드를 포함하고 있으나, avrdude의 -c usbasp 모드는 HVSP를 지원하지 않으므로 별도의 Python 호스트 도구가 필요합니다.

6. 12V HVSP 모드 상세

ATtiny85의 HVSP 프로토콜은 11비트 데이터/명령 프레임을 4개 핀(SDI, SDO, SII, SCI)으로 직렬 전송합니다. 진입 시퀀스는 다음과 같습니다:

1. 타겟 VCC 인가 (5V, 1.8V 이상에서 20us 이내 상승)
2. SDI/SII/SDO 라인을 LOW 로 유지
3. RESET 을 LOW 로 설정 (0V)
4. 12V 를 RESET 핀에 인가 (11.5 ~ 12.5V)
5. 최소 10us 대기
6. SDO 를 입력으로 전환 (드라이브 충돌 방지)
7. 최소 300us 대기 후 직렬 명령 전송 시작

12V 생성은 외부 부스트컨버터가 필요합니다. MAX662A (12V 30mA 전용 IC, 가장 간단) 또는 MC34063A + 인덕터 + 다이오드 조합을 사용할 수 있습니다. RP2040-Zero의 GP10 출력으로 부스트컨버터의 EN (또는 SHDN) 핀을 제어하여 HVSP 모드 진입 시에만 12V를 인가합니다 (v1.9 에서 12V_EN 이 GP11 에서 GP10 으로 이동했습니다). 12V는 RESET 핀에만 인가되어야 하며, VCC는 5V를 유지해야 합니다.

HVSP 핀 할당 (RP2040-Zero, v1.9: MOSI/SDI 공유)

Pico 핀	HVSP 신호	ATtiny85 핀	방향	설명
GP3	SDI (공유)	PB1 (6)	출력	데이터 입력, ISP MOSI 와 동일 핀 공유 (v1.9)
GP7	SCI	PB0 (5)	출력	HVSP 클럭
GP8	SDO	PB2 (7)	입력	데이터 출력 (타겟 -> Pico)
GP9	SII	PB3 (2)	출력	명령어 입력
GP10	12V_EN	부스트 ON	출력	12V 부스트컨버터 인에이블
GP11	(free)	-	-	v1.9 에서 사용자 자유 핀으로 반환

표 4. HVSP 핀 할당 (RP2040-Zero, v1.9 MOSI/SDI 공유)

RP2040-Zero 핀 선택 이유: GP16 은 온보드 WS2812 RGB LED 에 연결되어 있어 일반 GPIO 로 사용하기 어렵습니다. GP22-GP25 는 바닥면 솔더 패드로만 접근 가능하여 빵판 실험에 부적합합니다. v1.9 부터는 ISP MOSI (GP3) 와 HVSP SDI 를 동일한 핀에 공유합니다. 두 신호는 모두 Pico -> 타겟 출력이며 MSB 우선 직렬 전송이고, ISP 와 HVSP 는 동시에 활성화되지 않으므로 충돌이 없습니다. HVSP 신호는 GP3 (SDI 공유) + GP7-GP10 (SCI/SDO/SII/12V_EN) 에 할당되며, GP11 은 사용자 자유 핀으로 반환됩니다.

HVSP 사용법 (Python 호스트 도구)

avrdude 의 USBasp 드라이버는 HVSP 명령을 지원하지 않으므로, 본 프로젝트는 별도의 Python 호스트 도구 `scripts/usbasp_pico_hvsp.py` 를 제공합니다. 이 도구는 pyusb 를 통해 USB vendor control transfer (요청 코드 200-207)를 전송하여 펌웨어의 HVSP 백엔드를 구동합니다.

```
pip install pyusb

# 시그니처 읽기 (12V 진입 검증)
python3 usbasp_pico_hvsp.py readsig

# 모든 퓨즈 읽기
python3 usbasp_pico_hvsp.py readfuses

# 칩 지우기 (락 비트도 클리어)
python3 usbasp_pico_hvsp.py erase

# 퓨즈 쓰기 (예: ATtiny85 기본값)
python3 usbasp_pico_hvsp.py writefuse low 0x62
```

```
python3 usbasp_pico_hvsp.py writefuse high 0xD7
python3 usbasp_pico_hvsp.py writefuse ext 0xFE

# 플래시 256 바이트 덤프
python3 usbasp_pico_hvsp.py dumpflash 0 256 > flash.bin
```

HVSP vendor 요청 코드 (USBasp-Pico 전용): 본 프로젝트는 원본 USBasp 프로토콜과의 호환성을 유지하기 위해, 표준 USBasp 요청 (1-16, 127)은 그대로 두고 HVSP 전용 요청을 200-207 범위에 추가했습니다. avrdude -c usbasp 플로우는 이 범위를 접근하지 않으므로 일반 ISP/TPI 프로그래밍은 영향받지 않습니다.

코드	이름	방향	용도
200	HVSP_CONNECT	OUT	12V 인가 + HVSP 모드 진입
201	HVSP_DISCONNECT	OUT	12V 차단 + 핀 해제
202	HVSP_CHIP_ERASE	OUT	칩 지우기 (락 비트 포함)
203	HVSP_READ_SIG	IN	3바이트 시그니처 읽기
204	HVSP_READ_FUSE	IN	퓨즈 읽기 (low/high/ext)
205	HVSP_WRITE_FUSE	OUT	퓨즈 쓰기 (low/high/ext)
206	HVSP_READ_FLASH	IN	플래시 1바이트 읽기
207	HVSP_WRITE_FLASH	OUT	플래시 16비트 워드 쓰기

표 5. USBasp-Pico HVSP vendor 요청 코드

6.5 SWIO 모드 (CH32V003 RISC-V)

본 프로젝트는 WCH CH32V003 RISC-V MCU 의 단선 디버그/프로그래밍 인터페이스인 SWIO (Single-Wire I/O) 를 추가로 지원합니다. SWIO 는 USB CDC ACM 가상 시리얼 포트를 통해 ASCII 명령으로 구동됩니다. avrdude 와는 별도 경로이므로 USBasp 호환성에 영향을 주지 않습니다.

SWIO 핀 할당: GP12 (RP2040-Zero 엣지 핀, ISP/HVSP 와 충돌 없음). SWIO 라인은 오픈드레인이므로 외부 4.7k~10k 풀업 저항이 필요합니다. CH32V003 는 3.3V 동작이므로 Pico 3V3 핀에서 직접 전원을 공급할 수 있습니다.

SWIO 프레임 포맷

SWIO 는 1비트 시작(0) + 7비트 레지스터 주소 (MSB 우선) + 1비트 R/W (0=읽기, 1=쓰기) + 32비트 데이터 (LSB 우선) 로 구성된 35비트 프레임을 사용합니다. RP2040 GPIO bit-bang 으로 ~125 kHz 클럭을 생성합니다.

CDC ASCII 명령 프로토콜

명령	인자	응답	설명
?	없음	+	CDC 통신 테스트
i	없음	!	SWIO 핀 초기화
s	없음	+	SWIO 리셋 (50 high clocks)
p	없음	+	타겟 전원 ON (SWIO idle high)
P	없음	+	타겟 전원 OFF
r	reg	4 hex bytes + "\n"	WCH 디버그 레지스터 읽기
w	reg val	+	WCH 디버그 레지스터 쓰기

표 6. SWIO CDC ASCII 명령 프로토콜

Python 호스트 도구 사용법

```
pip install pyserial

# CDC 포트 확인 (Linux: /dev/ttyACM0, macOS: /dev/cu.usbmodem*)
ls /dev/ttyACM*

# 연결 테스트
python3 scripts/usbasp_pico_swio.py --port /dev/ttyACM0 ping

# SWIO 초기화
python3 scripts/usbasp_pico_swio.py --port /dev/ttyACM0 init

# 디버그 레지스터 0x06 읽기 (데이터 전송 레지스터)
python3 scripts/usbasp_pico_swio.py --port /dev/ttyACM0 r 06

# 디버그 레지스터 0x7F = 0x12345678 쓰기
python3 scripts/usbasp_pico_swio.py --port /dev/ttyACM0 w 7F 12345678

# SWIO 리셋 (50 high clocks)
python3 scripts/usbasp_pico_swio.py --port /dev/ttyACM0 reset
```

minichlink 호환성 (CDC binary 모드)

CDC 인터페이스는 두 가지 프로토콜을 동시에 지원합니다 (첫 바이트로 자동 판별):

1) Binary 모드 (Ardulink 호환, minichlink 사용): 첫 바이트가 r/w/p/P 인 경우. 개행 문자 없이 고정 길이 바이너리 프레임.

명령	전송	응답	비고
r	0x72, reg7	4 bytes LE value	minichlink ReadReg32
w	0x77, reg7, u32 LE	0x2B ('+')	minichlink WriteReg32
p	0x70	0x2B ('+')	전원 ON / SWIO init

P	0x50	0x2B ('+')	전원 OFF

표 8. CDC binary 모드 명령 (Ardulink / minichlink 호환)

2) ASCII 모드 (터미널 / 우리 도구): 첫 바이트가 ?/i/s 인 경우. 줄바꿈으로 종료되는 텍스트 명령. ? -> "+\n", i -> "!\n" (SWIO init), s -> "+\n" (50 high clocks 리셋).

시작 시 동기 문자: 펌웨어 부팅 후 CDC 포트에 "!" (1 바이트, 개행 없음) 를 출력합니다. 이것은 minichlink 의 TryInit_Ardulink 가 기다리는 Ardulink 동기 문자입니다.

minichlink 직접 사용

CDC 포트가 Ardulink 프로토콜을 말하므로, minichlink 를 직접 연결하여 CH32V003 플래시 프로그래밍이 가능합니다. minichlink 의 -A (또는 --ardulink) 플래그로 Ardulink 모드 강제:

```
# minichlink 빌드 (ch32v003fun 저장소)
cd ch32v003fun/minichlink
make

# CDC 포트로 CH32V003 플래시 쓰기
minichlink -A -d /dev/ttyACM0 -w firmware.bin 0x08000000

# 칩 정보 확인
minichlink -A -d /dev/ttyACM0 -i

# 칩 지우기
minichlink -A -d /dev/ttyACM0 -E
```

참고: minichlink 버전에 따라 -A 대신 -C ardulink 또는 별도의 인터페이스 설정이 필요할 수 있습니다. 최신 minichlink 는 Ardulink 를 자동 감지하므로 -d 만으로도 동작할 수 있습니다.

본 도구는 두 가지 USB 전송 방식을 모두 지원합니다 (자동 감지):

1) Vendor request (pyusb, 기본): USBasp vendor control transfer 로 SWIO 명령 전송. USBasp-Pico 전용 요청 코드 210-213 사용. 별도 시리얼 포트 없이 VID/PID 만으로 장치 접근.

2) CDC ACM (pyserial): 가상 시리얼 포트 (/dev/ttyACM0) 로 ASCII 명령 전송. 사람 친화적 터미널 인터페이스.

코드	이름	방향	용도
210	SWIO_INIT	OUT	SWIO 핀 초기화 (오픈드레인, idle high)
211	SWIO_RESET_TARGET	OUT	50 high clocks (CH32V003 디버그 리셋)
212	SWIO_READ_REG	IN	4바이트 LE 값 응답 (wValue low = reg)
213	SWIO_WRITE_REG	OUT	4바이트 LE 값 데이터 스테이지 (wValue low = reg)

표 7. USBasp-Pico SWIO vendor 요청 코드 (USBasp 위 확장)

Python 도구 사용법 (vendor request 방식)

```
pip install pyusb

# SWIO 초기화
python3 scripts/usbasp_pico_swio.py init

# SWIO 타겟 리셋
python3 scripts/usbasp_pico_swio.py reset

# 디버그 레지스터 0x06 읽기
python3 scripts/usbasp_pico_swio.py r 06

# 디버그 레지스터 0x7F = 0x12345678 쓰기
python3 scripts/usbasp_pico_swio.py w 7F 12345678

# CDC 방식 강제 (시리얼 포트 경로 지정)
python3 scripts/usbasp_pico_swio.py --cdc --port /dev/ttyACM0 r 06
```

7. 빌드 방법

빌드에는 ARM Cortex-M0+ 크로스 컴파일러 (arm-none-eabi-gcc 10+), CMake 3.13+, Ninja, Pico SDK 2.x가 필요합니다. 빌드 과정은 표준 Pico SDK 패턴을 따릅니다.

필수 도구

arm-none-eabi-gcc (ARM 공식 다운로드 또는 배포판 패키지)

cmake 3.13 이상

ninja (권장) 또는 make

Pico SDK 2.3.0 이상 (raspberrypi/pico-sdk)

TinyUSB (Pico SDK 서브모듈로 자동 포함)

빌드 명령

```
# 환경 변수 설정
export PICO_SDK_PATH=/path/to/pico-sdk
export PATH=$PATH:/path/to/arm-none-eabi-gcc/bin

# Pico SDK 서브모듈 초기화 (최초 1회)
cd $PICO_SDK_PATH && git submodule update --init lib/tinyusb

# 빌드 디렉토리 생성 및 구성
cd /home/z/my-project/usbasp-pico
cmake -B build -G Ninja

# 컴파일 및 UF2 생성
cmake --build build

# 결과물
# build/usbasp-pico.uf2 (플래싱용)
# build/usbasp-pico.elf (디버그용)
# build/usbasp-pico.hex (일반 헥사)
```

Pico 플래싱

1. Pico의 BOOTSEL 버튼을 누른 채 USB 연결 (대용량 저장장치로 인식)
2. build/usbasp-pico.uf2 파일을 Pico 드라이브에 복사
3. Pico가 자동 리부팅되며 USBasp로 인식

HVSP 비활성화 빌드: CMakeLists.txt에서 USBASP_PICO_ENABLE_HVSP=1을 0으로 변경하면 HVSP 관련 코드가 스텁 처리되어 플래시 사용량이 줄어듭니다. 12V 회로를 구성하지 않은 경우 추천합니다.

8. avrdude 호환성 및 사용법

USBasp-Pico는 표준 USBasp로 인식되므로 avrdude의 -c usbasp 옵션으로 그대로 사용할 수 있습니다. avrdude 5.2 이상에서 USBasp가 지원됩니다.

OS별 드라이버 설정

```
Linux: udev 룰 추가 (일반 사용자 권한)
sudo nano /etc/udev/rules.d/99-usbasp.rules
SUBSYSTEM=="usb", ATTRS{idVendor}=="16c0", ATTRS{idProduct}=="05dc", MODE="0666", GROUP="plugdev"
sudo udevadm control --reload-rules
```

Windows: Zadig (https://zadig.akeo.ie/) 실행 후 USBasp 장치를 libusb-win32 또는 WinUSB 드라이버로 설정합니다. libusbK도 호환됩니다.

macOS: libusb (Homebrew: `brew install libusb`) 설치 후 별도 드라이버 불필요.

avrdude 명령 예제

```
# ATmega328P 시그니처 읽기
avrdude -c usbasp -p m328p

# 플래시 쓰기 + 검증
avrdude -c usbasp -p m328p -U flash:w:firmware.hex

# 퓨즈 읽기
avrdude -c usbasp -p m328p -U lfuse:r:-:h -U hfuse:r:-:h

# 퓨즈 쓰기 (예: 내부 8MHz 클럭)
avrdude -c usbasp -p m328p -U lfuse:w:0xE2:m

# SCK 속도 지정 (느린 클럭 AVR)
avrdude -c usbasp -p m328p -B 8 # 8 kHz SCK

# TPI 모드 (ATtiny10)
avrdude -c usbasp -p t10 -U flash:w:tiny10.hex
```

9. 테스트 및 검증

펌웨어 정상 동작 확인 절차는 다음과 같습니다. 빌드 후 UF2 플래싱, USB 인식, avrdude 통신, 실제 AVR 프로그래밍 단계로 나뉩니다.

검증 체크리스트

단계	확인 항목	성공 기준
1	UF2 파일 크기	70KB 이상 (실제 펌웨어 ~38KB)
2	USB 인식	lsusb 출력에 16c0:05dc 표시
3	avrdude 인식	avrdude -c usbasp -p m328p 시 "found USBasp"
4	시그니처 읽기	m328p: 0x1E 0x95 0x0F 일치
5	플래시 쓰기	4KB 블록 단위 정상 전송 (avrdude 진행률 표시)
6	플래시 검증	-U flash:v:firmware.hex 정상 완료
7	퓨즈 읽기/쓰기	lfuse/hfuse/efuse 정상 읽기 + 쓰기
8	EEPROM 접근	EEPROM 읽기/쓰기 정상 동작
9	고용량 AVR	ATmega2560 256KB 플래시 정상 프로그래밍
10	TPI 모드	ATtiny10 시그니처 0x1E 0x90 0x07 정상

표 5. 펌웨어 검증 체크리스트

문제 해결

avrdude가 USBasp를 찾지 못함: USB 케이블/포트 확인, udev 룰 적용 여부, 드라이버 설치 상태 확인. lsusb로 16c0:05dc가 보이는지 먼저 점검.

시그니처 0x00 0x00 0x00 읽힘: SCK 속도가 너무 빠름. -B 32 또는 -B 16으로 느리게 설정 후 재시도. 타겟 클럭이 1.5MHz 이하인 경우 필수.

플래시 검증 실패: 배선 불량(MISO 특히), GND 공통 누락, 타겟 VCC 부족(500mA 이상 필요 시 외부 전원).

고용량 AVR 실패: avrdude 버전 5.11+ 필요 (SETLONGADDRESS 지원). 구버전에서는 128KB 이상 주소 접근 불가.

10. 관련 프로젝트 및 참고자료

본 프로젝트는 다음 오픈소스 프로젝트들을 기반으로 하거나 참고했습니다. 각각의 접근 방식과 본 프로젝트의 차이점을 정리합니다.

프로젝트	구현 방식	본 프로젝트와 차이
USBasp (Thomas Fischl)	ATmega8 + V-USB, 12MHz	원본. 본 프로젝트의 프로토콜 기반
mariusgreuel/USBasp	USBasp 포크 (윈도우 드라이버/빌드 개선)	본 프로젝트의 직접 베이스
USBtinyISP (Dick Streefland)	ATtiny2313 + V-USB	고용량 AVR 미지원, 본 프로젝트의 대안
SpeedyCraftah/avr-pico-programmer	Pico + CDC 시리얼 + 자체 프로토콜	avrdude 비호환, Node.js 호스트 필요
kevincuzner/linuxspi	Raspberry Pi (Pi 4 등) + linux SPI 드라이버	RP2040 아님, avrdude -c linuxspi
tsaarni/avr-hvsp	Arduino Nano + HVSP (ATtiny85)	USB 비호환, HVSP 전용
ericdraken/hvsp	ATtiny85 + HVSP charge pump 자체 PCB	RP2040 아님, HVSP 참고용
AVRDUDE (avrdudes/avrdude)	USBasp 호스트 드라이버 구현체	펌웨어와 호스트 매칭 기준

표 6. 관련 프로젝트 비교

참고 자료

USBasp 원본: https://www.fischl.de/usbasp/

mariusgreuel 포크: https://github.com/mariusgreuel/USBasp

AVRDUDE: https://github.com/avrdudes/avrdude

Raspberry Pi Pico SDK: https://github.com/raspberrypi/pico-sdk

TinyUSB: https://github.com/hathach/tinyusb

ATtiny85 datasheet (HVSP): https://www.microchip.com/wwwproducts/ATtiny85

MAX662A 12V charge pump: https://www.analog.com/max662

V-USB (원본 USB 드라이버): https://www.obdev.at/products/v-usb/

11. 라이선스 및 크레딧

본 프로젝트의 펌웨어 소스 코드는 USBasp 원본의 GNU GPL v2 라이선스를 계승합니다. USBasp 프로토콜 사양 및 프로토콜 상수(USBASP_FUNC_*, USBASP_ISP_SCK_*)는 Thomas Fischl의 원본 USBasp에서 파생되었습니다. mariusgreuel 포크의 개선사항(빌드 시스템, 드라이버 패키징)을 참고했습니다.

사용된 오픈소스 컴포넌트

컴포넌트	라이선스	저작권
USBasp (Thomas Fischl)	GPL v2	Thomas Fischl (2005~2011)
mariusgreuel/USBasp fork	GPL v2	Marius Greuel
V-USB	GPL v2 / 상용	OBJECTIVE DEVELOPMENT Software GmbH
AVRDUDE	GPL v2	avrdudes 커뮤니티, Dick Streefland
TinyUSB	MIT	Ha Thach (tinyusb.org)
Pico SDK	BSD-3-Clause	Raspberry Pi (Trading) Ltd
arm-none-eabi-gcc	GPL v3 + GCC runtime exception	Free Software Foundation

표 7. 사용된 오픈소스 컴포넌트 라이선스

배포 조건: 본 펌웨어 바이너리(usbasp-pico.uf2)는 GPL v2 조건에 따라 배포됩니다. 소스 코드 요청 시 본 저장소(또는 미러)를 통해 제공되며, 수정된 버전을 배포할 경우 동일한 GPL v2 라이선스를 유지해야 합니다.

면책 조항: 본 프로젝트는 어떠한 보증도 제공하지 않습니다. 12V HVSP 모드 사용 시 타겟 AVR 손상 위험이 있으므로, 데이터시트를 충분히 숙지한 후 사용하시기 바랍니다. 특히 RESET 핀에 12V 이상의 전압이나 부정극성 전압이 인가되지 않도록 주의해야 합니다.